

MASTER PLAN • MULTIMODAL UNDERSTANDING

Perception SLM

Small, from-scratch models that **understand** images, video, and audio — not generate them. On-device first, cloud second. Efficiency is the moat.

0.3-3B params

Image

Video

Audio

Not generation

HuggingFace-first data

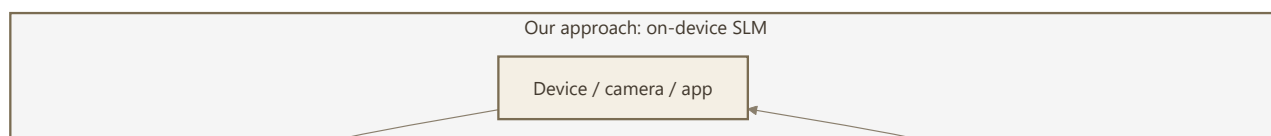
- 01 The problem
- 02 Pros & cons
- 03 Architecture
- 04 Components
- 05 Tech stack
- 06 System design
- 07 Issues & optimization
- 08 Datasets
- 09 Roadmap
- 10 Repo & tests

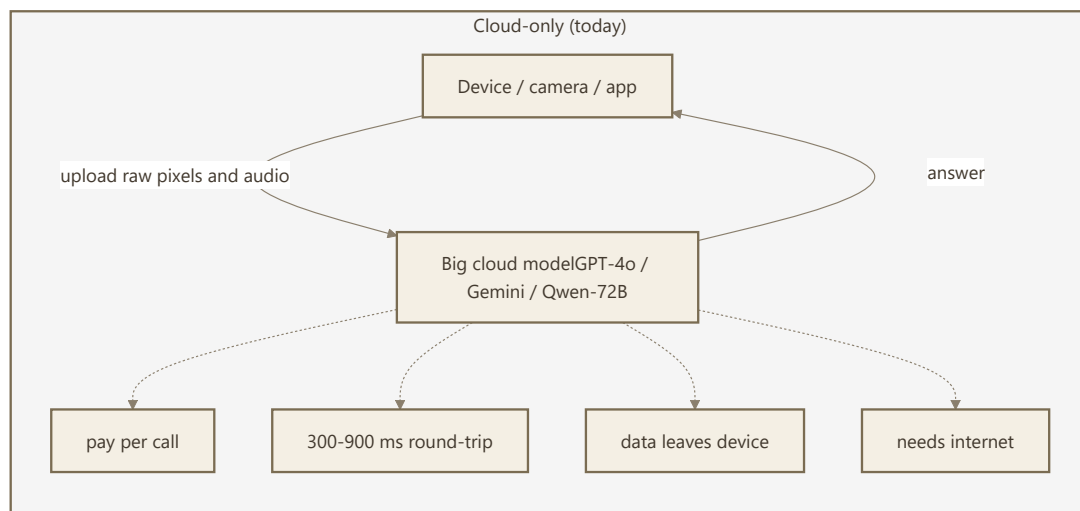
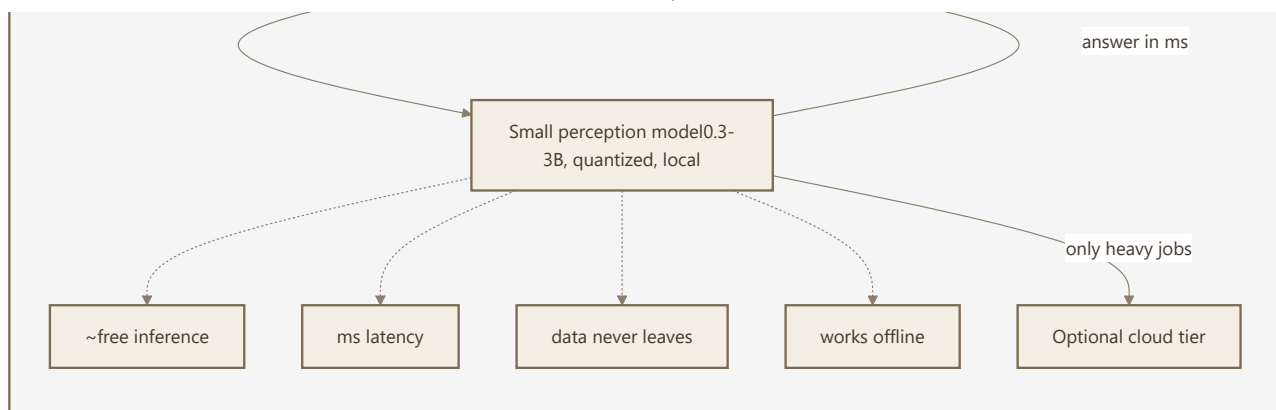
SECTION 01

The problem — what we solve

Understanding an image, video, or audio clip today means calling a **giant cloud model**. For a real product that means three hard problems: **cost** (pay per call), **latency + connectivity** (round-trips, no offline), and **privacy** (user data leaves the device). Small on-device models remove all three at once.

Fig 1 • Cloud-only vs on-device perception





SECTION 02

Pros & cons

You don't win on model size — you win on **proprietary task data** and being **the best at running tiny + on-device**. Everything downstream optimizes for those two.

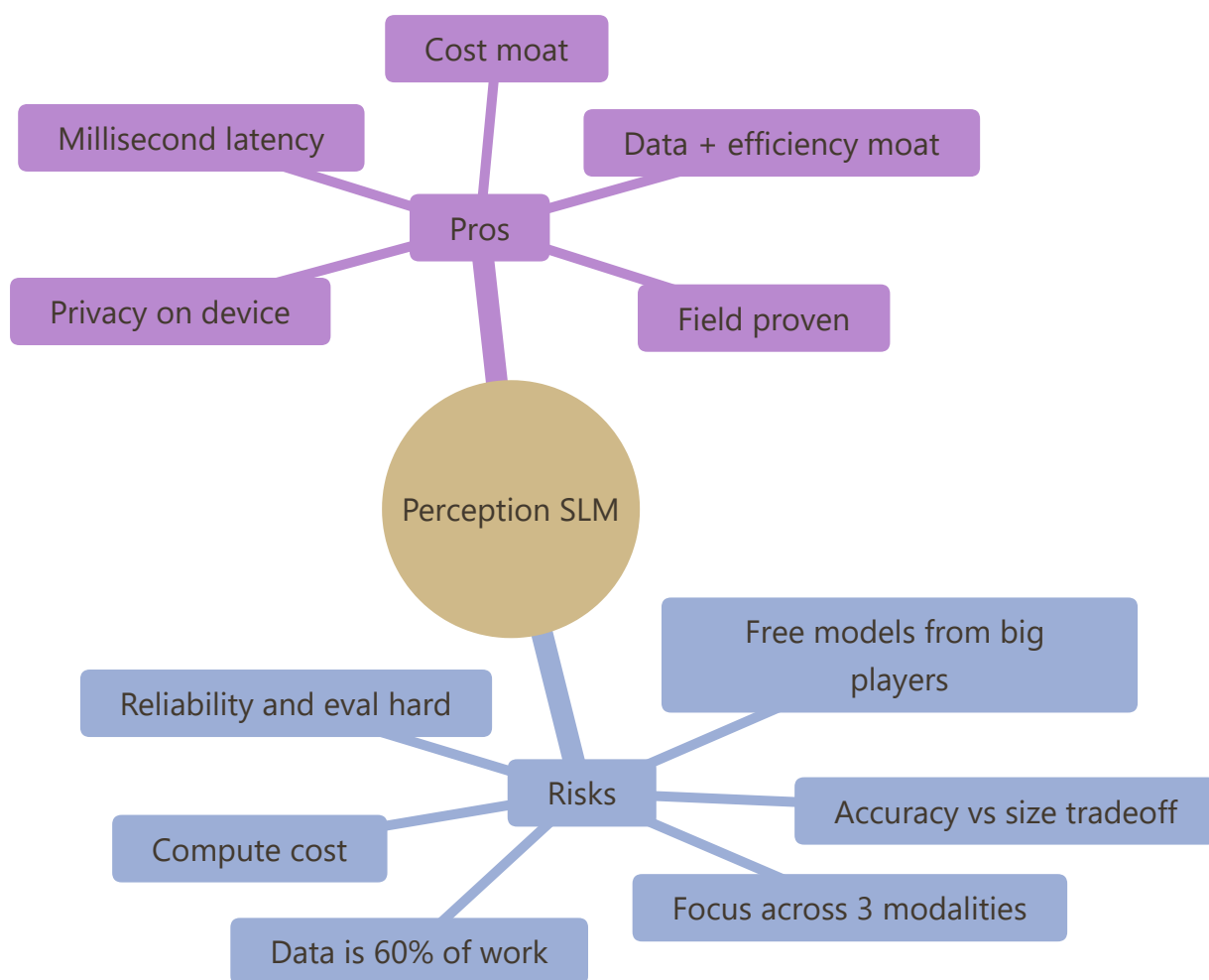
PROS

- Cost moat — inference ~free vs per-call APIs
- Privacy — data stays on device (health, finance)
- Latency — ms → real-time video, robotics, AR
- Defensible via **data + efficiency**, not scale
- Field-proven: SmolVLM, Moondream, Whisper
- From-scratch is realistic at small size

CONS / RISKS

- 3 modalities = 3 startups → focus needed
- Data quality/labeling/licensing = 60% of work
- Small models trade accuracy for size
- Train compute real: 8–64 GPU-days / iteration
- Big players ship free small models
- Reliability & eval are genuinely hard

Fig 2 • Decision map

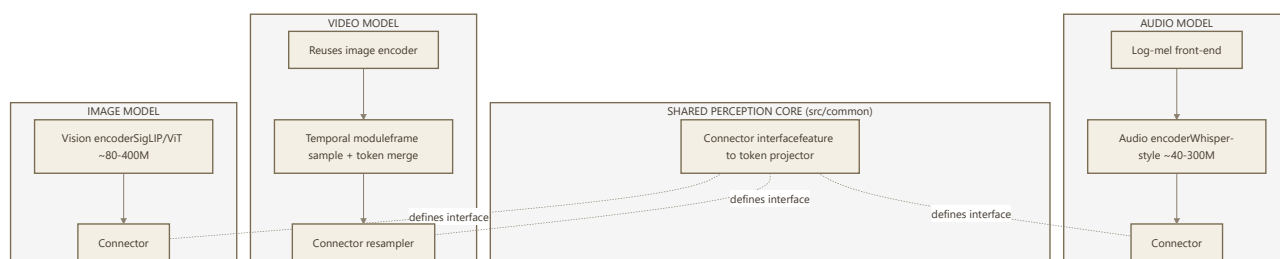


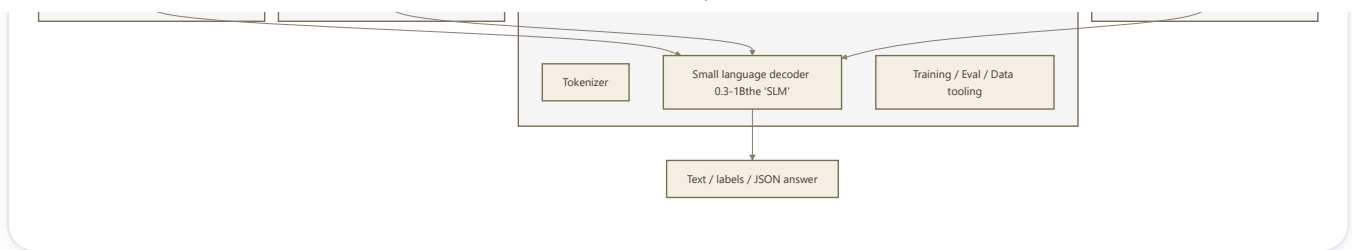
SECTION 03

Architecture — big picture

Three separate models, one shared core. Never force one backbone across pixels + frames + sound — it hurts quality. Share the **decoder + tooling**, not the encoders. What's genuinely *from scratch* is the **connector** (and optionally the decoder).

Fig 3 • Shared core + three models





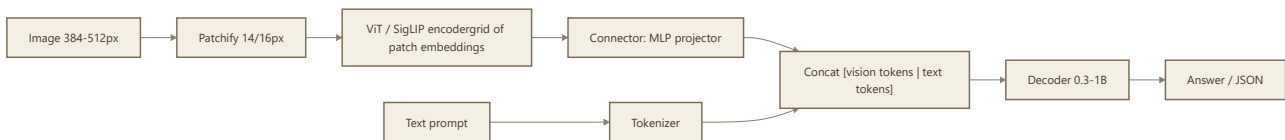
SECTION 04

Each component in depth

4.1 • Image model — data flow

Encoder ~80–400M params, 384–512px input, patch 14/16. From scratch via SigLIP contrastive pretrain, or start from open weights + finetune. Built **first** — its encoder is reused by video.

Fig 4 • Image — image + prompt → answer



4.2 • Video model — the time dimension

This is where efficiency work concentrates: $N \text{ frames} \times M \text{ patches}$ explodes token count. Fix with frame subsampling, keyframe selection, token merging, and a resampler.

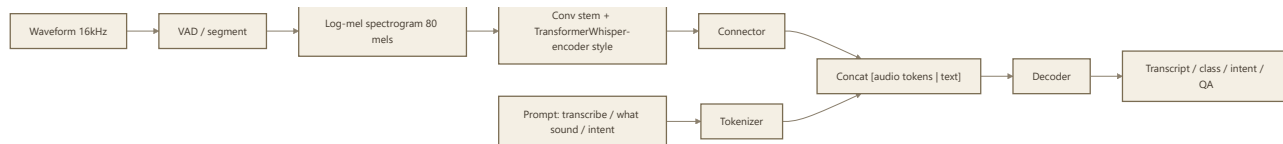
Fig 5 • Video — reuses image encoder + temporal module



4.3 • Audio model — independent pipeline

Understanding, not just transcription: sound classes, emotion, intent, audio Q&A. Fully independent → can be built **in parallel** from day one.

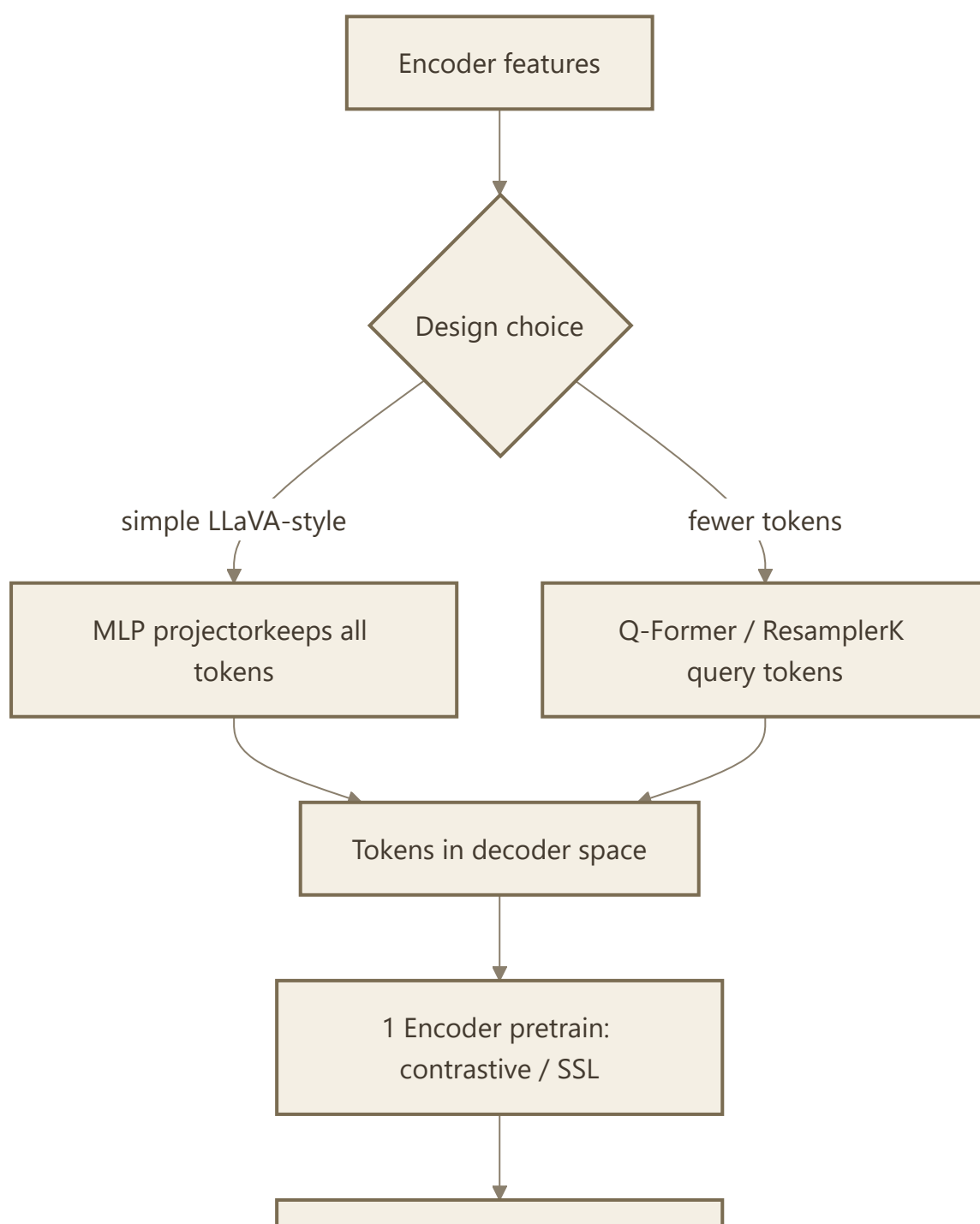
Fig 6 • Audio — waveform → meaning

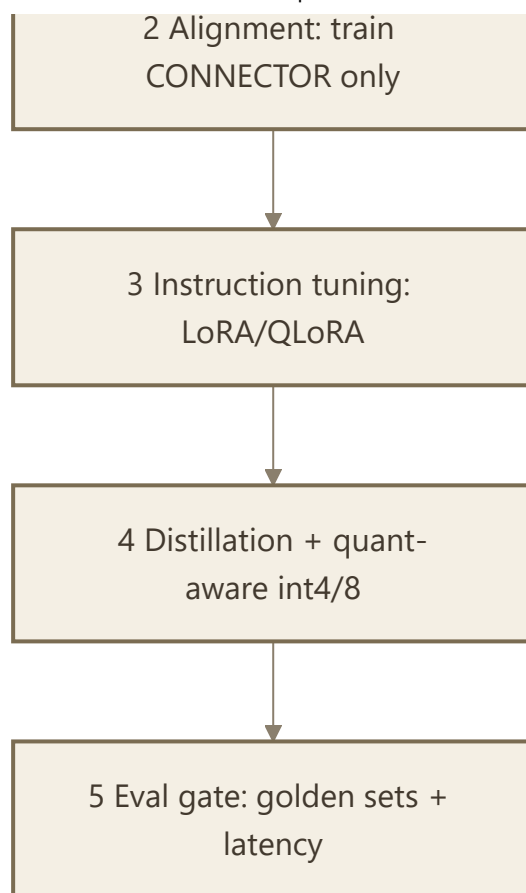


4.4 • Connector & training curriculum

The connector maps encoder features into the decoder's token space — trained on **our** (modality, text) pairs. That's the defensible asset. Training runs as a staged curriculum.

Fig 7 • Connector design + 5-stage training



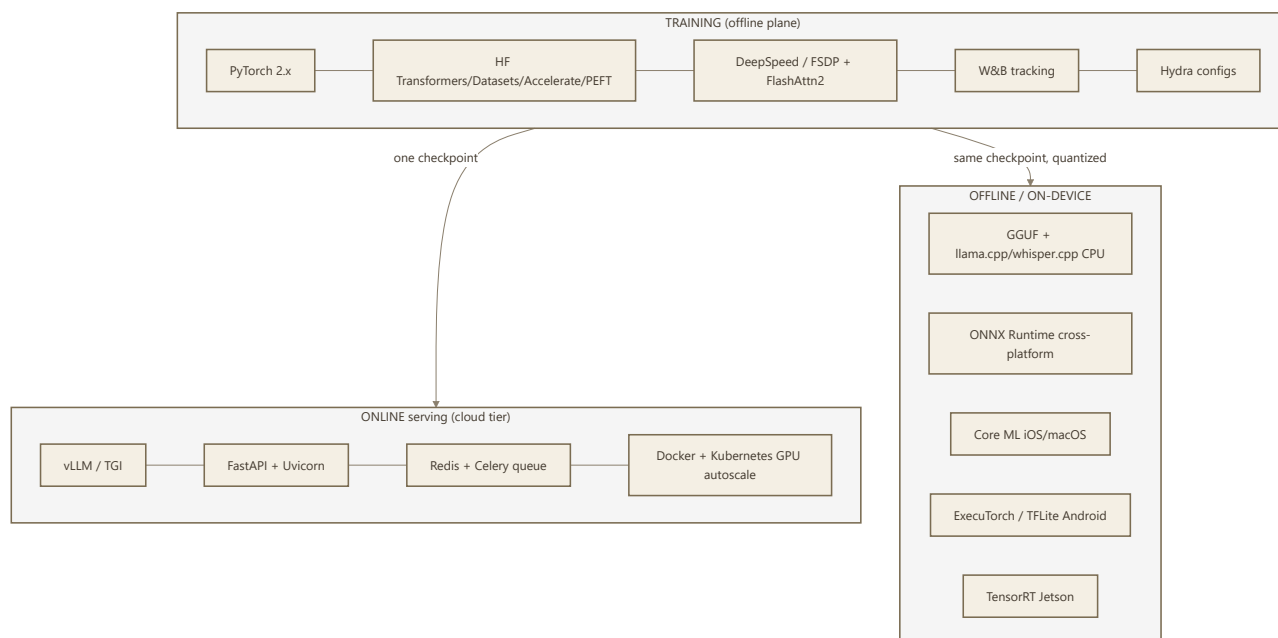


SECTION 05

Tech stack — offline / online / cloud

One training checkpoint produces two builds: a batched GPU server for the cloud tier, and a quantized single-file build for on-device. Never two codebases.

Fig 8 · Three planes — one checkpoint, two builds



Quant int4/8:
AWQ/GPTQ/GGUF

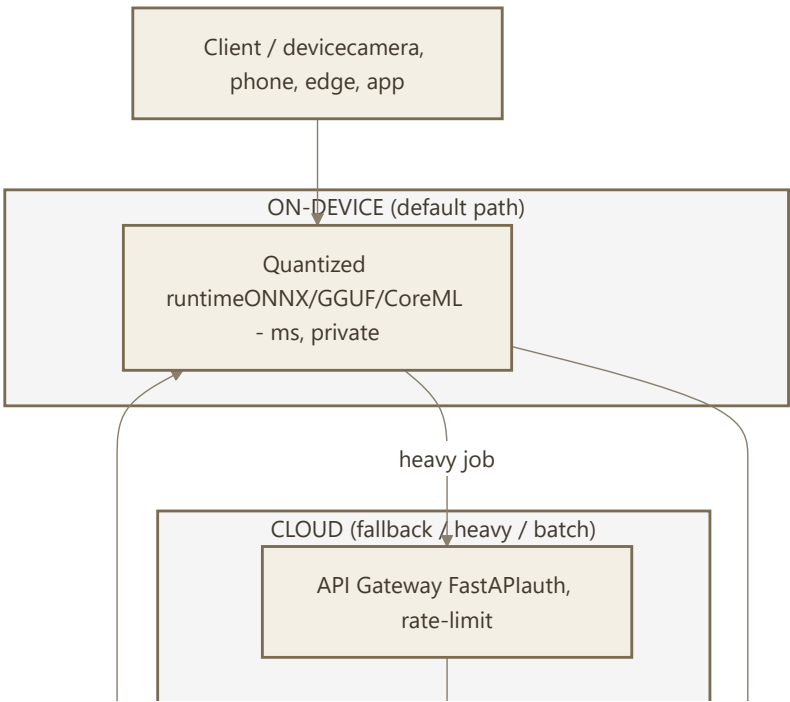
LAYER	OFFLINE / ON-DEVICE	ONLINE (CLOUD)	CLOUD INFRA
Runtime	GGUF, ONNX, Core ML, ExecuTorch, TensorRT	vLLM / TGI	RunPod/Lambda → AWS/GCP
Quantize	int4/int8 (AWQ, GPTQ, GGUF)	fp16 / int8	—
Storage	on-device file	—	S3 / GCS
Data source	—	—	HuggingFace Hub (streamed)
CI/CD	—	GitHub Actions → test + build	Model registry (HF private)

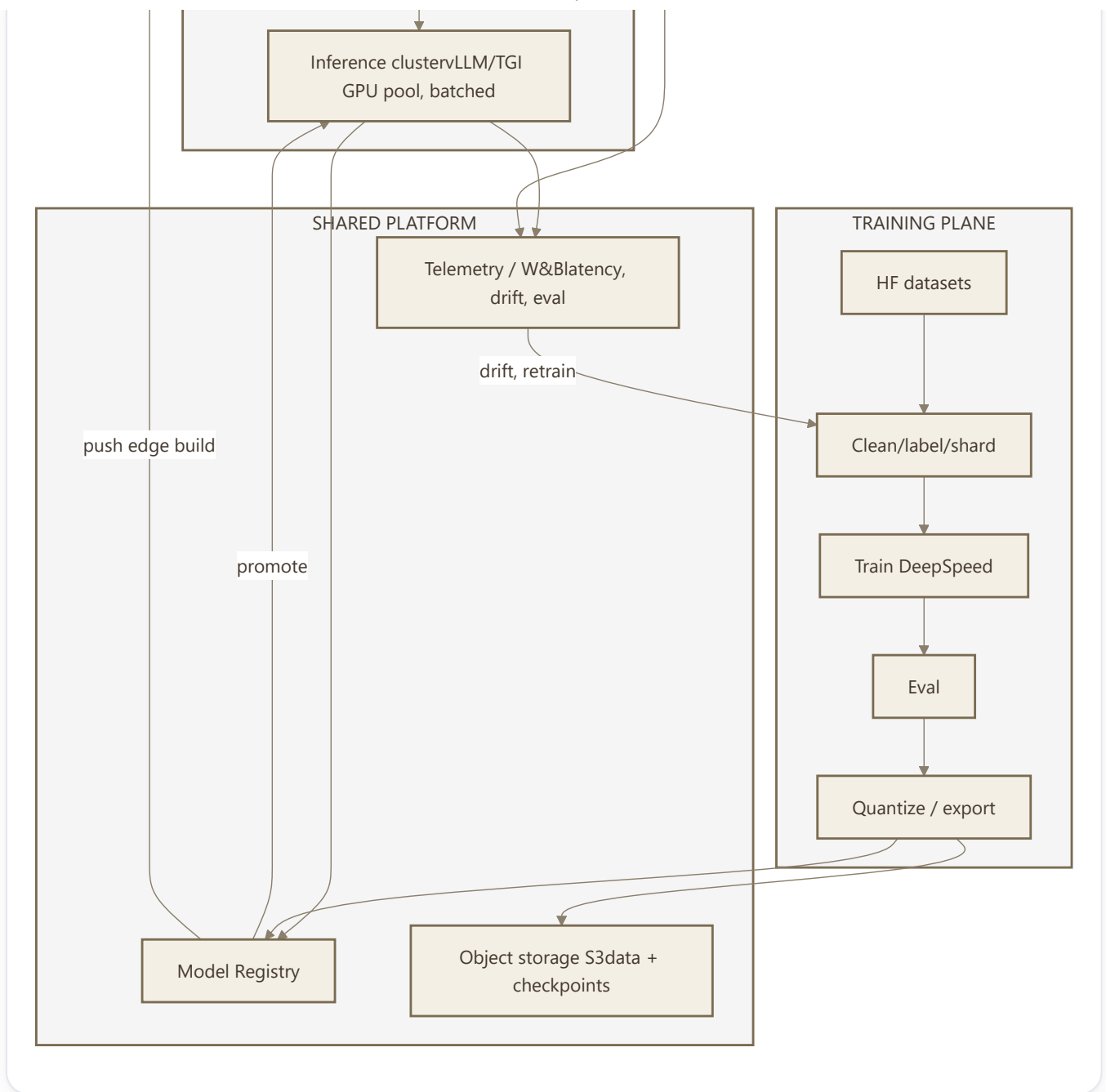
SECTION 06

System design — deployment planes

Device-first, cloud-fallback. Data version + model version + eval report always travel together. Drift telemetry feeds back into retraining.

Fig 9 · End-to-end system with drift → retrain loop

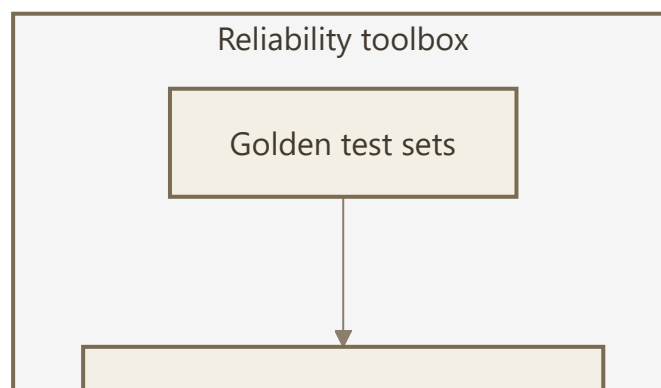


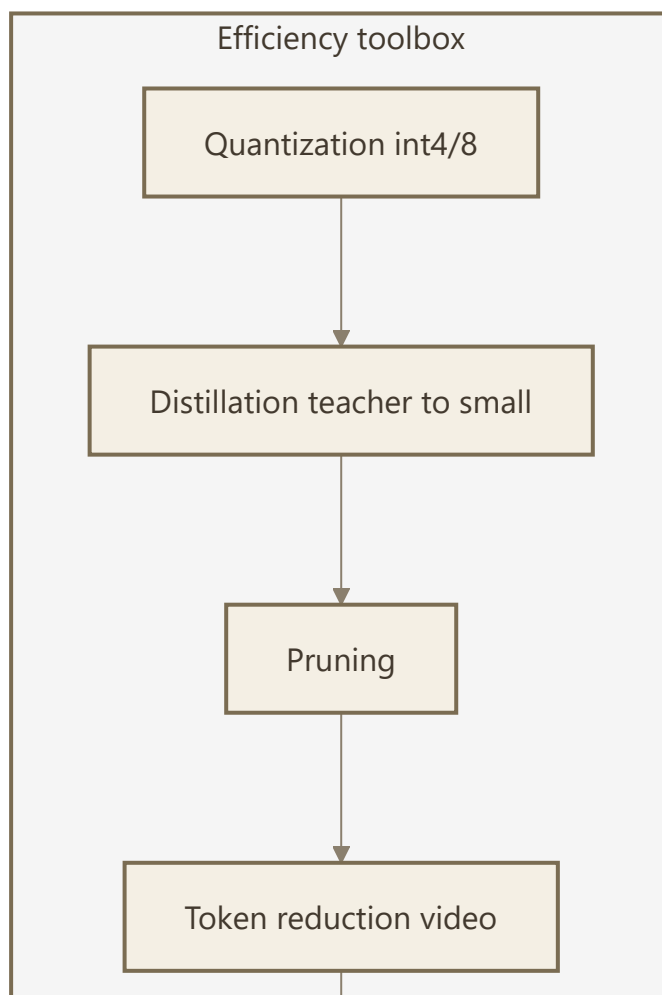
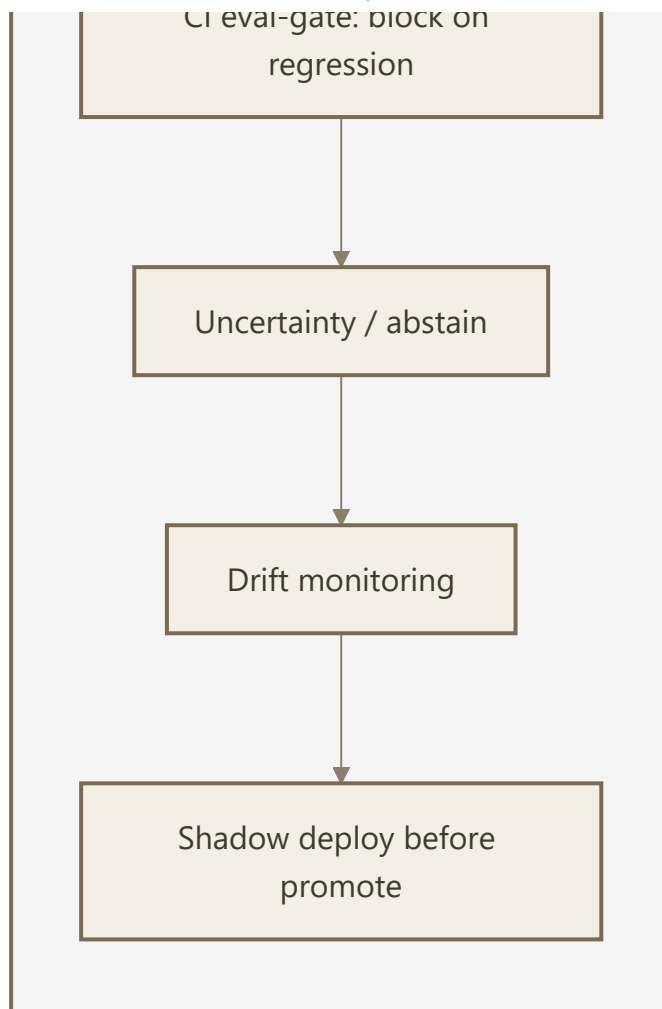


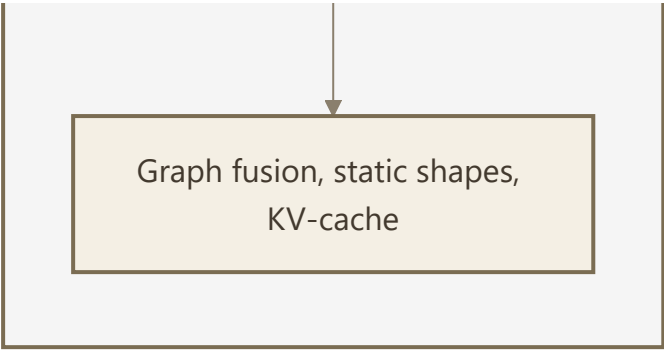
SECTION 07

Issues & how we optimize

Fig 10 · Efficiency & Reliability toolboxes (ranked)







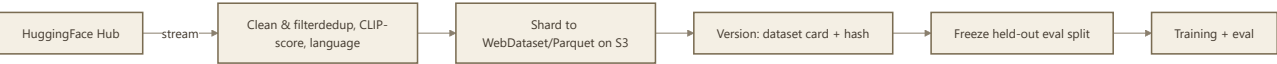
PROBLEM	SYMPTOM	FIX
Video token explosion	OOM, slow	frame subsample, keyframe, token merge (ToMe), resampler
Quantization accuracy drop	int4 gets dumb	quant-aware training, mixed 4/8-bit, per-channel, keep sensitive layers high-precision
Hallucination	invents unseen text	grounding loss, hard negatives, abstain training, constrained/JSON decode
Small model underfits	mediocre everywhere	distillation from big teacher, curriculum (easy→hard)
Data noise	unstable training	dedup, CLIP-score filter, consensus labels, clean held-out set
Field drift	demo works, field fails	collect field data, drift telemetry, periodic retrain
Weak-HW latency	too slow on phone	distill smaller, ONNX graph opt, fusion, static shapes
Audio robustness	fails on noise/accents	SpecAugment, diverse data, VAD front-end

SECTION 08

Datasets — HuggingFace-first

Stream from the Hub (don't fully download), clean and filter, shard, version, and freeze a held-out eval split per version. Always verify **licenses** before commercial use.

Fig 11 · Data pipeline



MODALITY	PRETRAIN / ALIGN	INSTRUCTION / TASK
Image	LAION subsets, COCO Captions, CC3M/CC12M, SBU, DataComp	LLaVA-Instruct, VQAv2, GQA, TextVQA, DocVQA, OCR-VQA, ChartQA, AI2D, FUNSD
Video	WebVid, HowTo100M	MSR-VTT, ActivityNet-Captions, NExT-QA, VATEX, Something-Something-v2, Ego4D
Audio	LibriSpeech, Common Voice, GigaSpeech, VoxPopuli, People's Speech	AudioSet, ESC-50, FSD50K, UrbanSound8K, AudioCaps, Clotho, WavCaps

SECTION 09

Roadmap — install → deploy

Audio is independent → start in parallel immediately. Video needs the image encoder → build Image first (fast) so Video reuses it. You still get audio + video early.

Fig 12 · Timeline (weeks)

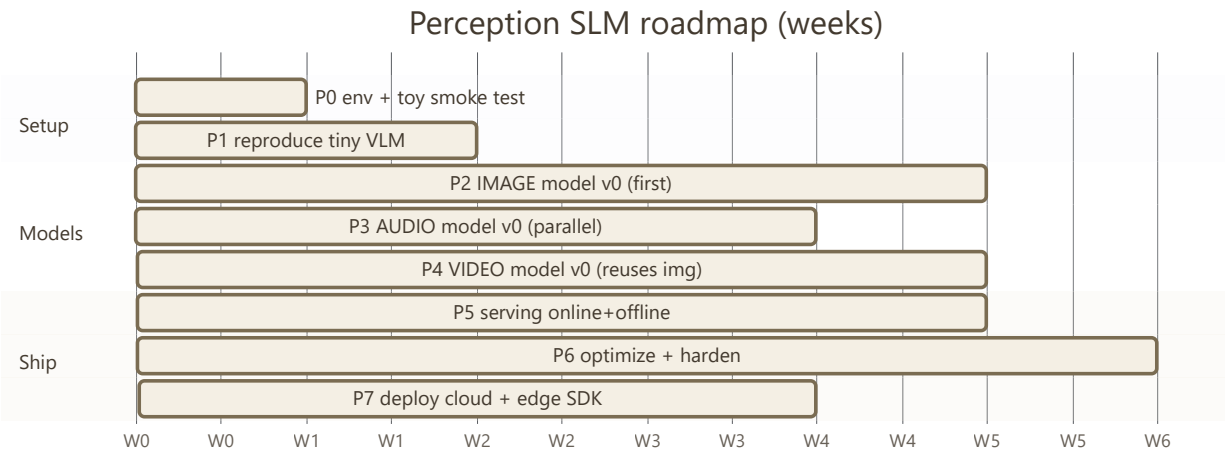
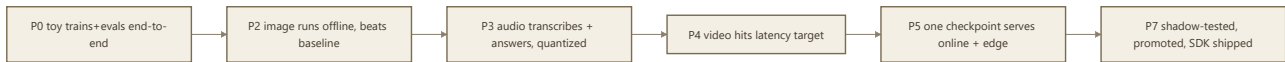


Fig 13 · Phase gates — must pass to advance

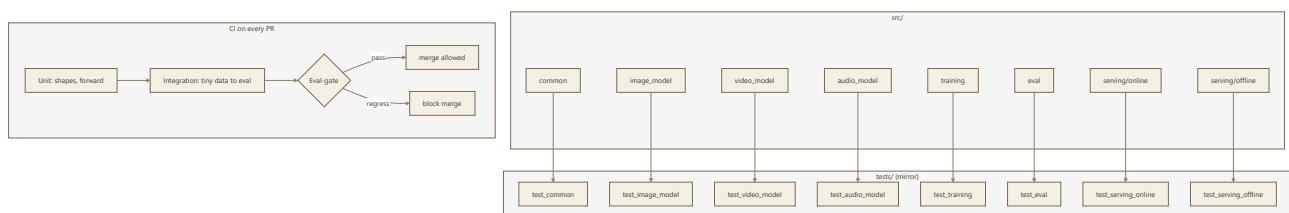


SECTION 10

Repo & test structure

Every functional section has its own source folder and a **mirrored test folder** (`test_<name>`). Tests are first-class — reliability is the product.

Fig 14 · `src` ↔ `tests` mirror + CI test tiers



```

img_video_audio/
├─ MASTER_PLAN.md · README.md · requirements.txt · .gitignore
├─ configs/                # Hydra/YAML per model & experiment
├─ data/{raw, processed, pipelines}
├─ src/
│   ├── common/            # tokenizer, connector, decoder, utils
│   ├── image_model/ video_model/ audio_model/
│   ├── training/ eval/
│   └─ serving/{online, offline}
├─ tests/test_{common,data_pipeline,image_model,video_model,
│   │           audio_model,training,eval,serving_online,serving_offline}/
├─ deploy/{docker, cloud, edge}
├─ notebooks/ scripts/
  
```

Next: scaffold starter code for `src/common` + `src/image_model` (encoder→connector→decoder skeleton + tests), then Phase 0 — environment + toy end-to-end smoke test.

